UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

Test Suite Minimization in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

As software systems continue to grow in size and complexity, the demand for extensive testing has become greater than ever. Consequently, the associated test suites tend to expand proportionally, often comprising thousands of individual test cases. While such comprehensive test suites enhance software reliability, they impose substantial costs in execution time, computational resources, and continuous integration latency [2]. Test suite minimization addresses this challenge by systematically eliminating redundant tests while preserving essential quality metrics, including fault-detection effectiveness and code coverage [19, 16]. However, naive reduction strategies risk removing high-value tests whose unique behavioral characteristics contribute to fault detection, thereby compromising software quality assurance.

This thesis investigates test suite minimization within the *Large-Scale Software Observatorium* (LASSO) [11], a platform designed for automated selection, execution, and comparative analysis of software systems at scale. LASSO integrates extensive open-source repositories and combines static analysis (code-level features) with dynamic analysis (execution behavior) into unified, language-agnostic analysis pipelines. This architecture enables empirically grounded studies of "big code" phenomena across diverse programming languages, with its current implementation primarily targeting Java-based systems.

Our approach leverages *Stimulus–Response Matrices* (SRMs) [11], LASSO's two-dimensional data representation for capturing system behavior by mapping test executions to their observed outcomes. Based on the resulting SRM, we can identify not only test failures but also extract coverage information and derive mutation values from corresponding mutated system variants. This abstraction enables minimization algorithms to operate uniformly across programming languages by analyzing system responses and execution outcomes instead of source code structures.

Throughout this thesis, we conduct a systematic literature review of several test suite minimization algorithms, covering greedy heuristics, exact optimization methods, search-based strategies, and data-driven techniques. Based on comparative evaluation across six criteria (coverage preservation, reduction effectiveness, fault-detection retention, computational scalability, SRM compatibility, and implementation feasibility), we identify the Mutation-Enhanced and Overlap-Aware Harrold–Gupta–Soffa (ME-OA-HGS) algorithm as the most suitable for integration into LASSO.

Our implementation of ME-OA-HGS within LASSO operates directly on SRM structures, enabling efficient test suite reduction while maintaining comprehensive coverage and robust fault-detection [5, 1]. Experimental validation on real-world SRMs demonstrated substantial reductions in test suite size while retaining the most critical tests, with the selection process consistently executing in polynomial time. The $O(nm \log m)$ complexity ensures scalability and makes the approach feasible for large-scale software analysis workflows.

Contents

List of Figures

List of Tables

List of Abbreviations

CFG .... Control Flow Graph
CI .... Continuous Integration
GA .... Genetic Algorithm
HGS .... Harrold–Gupta–Soffa algorithm
ILP .... Integer Linear Programming
LASSO .... Large-Scale Software Observatorium
LSL .... LASSO Scripting Language
ME-OA-HGS .... Mutation-Enhanced and Overlap-Aware HGS
QIGA .... Quantum-Inspired Genetic Algorithm
SRM .... Stimulus-Response Matrix
TSM .... Test Suite Minimisation

1. Introduction

Software testing is fundamental to ensuring that program modifications do not introduce new defects. As software systems grow in complexity, their associated test suites can become extremely large. Automated test generation tools such as Randoop [15] and EvoSuite [4] may produce thousands of test cases, with execution times potentially requiring days or weeks in large industrial projects [1]. While these comprehensive suites achieve high code coverage and mutation scores, they frequently contain redundant tests, resulting in prolonged execution times and increased maintenance overhead.

Test suite minimization addresses this challenge by systematically removing redundant tests while preserving the test suite's fault-detection effectiveness [19]. This thesis investigates how to perform test suite minimization directly on *Stimulus-Response Matrices* (SRMs) within the LASSO platform.

1.1. Background and Motivation

Regression testing serves as a critical quality assurance practice that verifies software modifications do not compromise existing functionality. As software systems evolve through iterative development cycles, regression test suites accumulate tests that validate previously implemented features, bug fixes, and behavioral specifications. This accumulation, while beneficial for quality assurance, creates significant practical challenges.

Software testing has emerged as one of the most significant cost factors in modern software development. Early empirical studies by Boehm [2] established that testing activities can consume nearly half of a software system's total development budget. Contemporary large-scale projects face even more acute challenges: automatically generated test suites may contain thousands or tens of thousands of test cases, requiring days or weeks to execute completely [16]. This escalating complexity directly impacts development velocity, infrastructure costs, and feedback latency in continuous integration pipelines.

Test-suite minimization addresses these challenges by systematically identifying and removing redundant tests while preserving essential quality metrics. The fundamental optimization objective is to reduce execution time and computational overhead without compromising the suite's ability to detect faults. However, this optimization presents a delicate balance: uninformed or aggressive reduction strategies can inadvertently remove high-value tests whose unique coverage patterns contribute disproportionately to fault detection, thereby undermining software quality assurance [19]. Consequently, effective minimization requires sophisticated algorithms that can identify genuine redundancy while preserving tests with distinct fault-revealing capabilities.

1.2. LASSO Context

The emergence of "big code" as a research paradigm has created unprecedented opportunities for empirical software engineering. However, mining software repositories at this scale introduces substantial technical and methodological challenges. Researchers must establish programmatically accessible software corpora containing millions of artifacts, construct efficient analysis infrastructures capable of processing these volumes, and develop precise metrics and extraction methodologies that achieve statistical significance and experimental repeatability.

While existing code mining platforms have successfully automated many analysis tasks at ultra-large scale, they predominantly focus on static properties derived from source code syntax and structure. This limitation excludes a crucial dimension of software behavior: dynamic properties observable only through program execution. Consequently, researchers cannot validate techniques that depend on runtime behavior, conduct experiments requiring execution traces, or measure performance characteristics that emerge only during actual program operation.

The Large-Scale Software Observatorium (LASSO) [11] addresses this fundamental gap by providing integrated infrastructure for both static and dynamic software analysis at scale. Developed at the University of Mannheim, LASSO features an extensive corpus of executable software systems aggregated from open-source repositories, coupled with a domain-specific language (DSL) for expressing selection criteria and analysis pipelines. This architecture enables researchers to formulate behavioral queries (e.g., "find implementations of sorting algorithms that exhibit $O(n \log n)$ performance") and execute them against thousands of candidate systems.

LASSO's key innovation lies in its systematic comparison methodology. The platform's "arena" component executes a common set of test stimuli against multiple software systems and captures their responses in a unified data structure called a *Stimulus-Response Matrix* (SRM) [11]. An SRM is a two-dimensional matrix where rows represent stimuli (test sequences, method invocations, or behavioral specifications), columns represent executable software systems under analysis, and cells contain structured response objects encoding execution results, coverage data, and observable behaviors. This abstraction enables purely data-driven comparison of functionally equivalent implementations across different programming languages without requiring source code access. Section 2.2 provides a formal definition of the SRM structure and details its role in the minimization workflow.

Despite LASSO's powerful capabilities for large-scale behavioral analysis, the platform currently lacks an integrated test-suite minimization component. This omission creates inefficiency in LASSO's execution pipelines: when analyzing hundreds or thousands of systems, redundant test stimuli unnecessarily prolong execution time, increase infrastructure costs, and delay experimental results. Integrating an effective minimization approach would enable LASSO to automatically reduce test redundancy while preserving the behavioral coverage necessary for meaningful cross-system comparison, thereby improving throughput without compromising analytical validity.

1.3. Problem Statement

Let $T = \{t_1, t_2, \ldots, t_n\}$ be a set of test cases and let $R = \{r_1, r_2, \ldots, r_m\}$ be the set of coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. Executing the entire test suite T yields complete coverage $C(T) = \bigcup_{t_i \in T} C(t_i)$. The *test suite minimization problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T)$. This optimization problem is equivalent to the classical set cover problem and is therefore NP-complete [19]. Consequently, practical approaches rely on heuristic algorithms and approximation techniques [5] that balance reduction effectiveness, coverage preservation, and computational efficiency.

Figure 1.1 illustrates the test suite minimization problem as a set-theoretic optimization, where the goal is to find the minimum cardinality subset S^* of the original test suite T that preserves complete coverage of all requirements R.

Original Test Suite

$T = \{t_1, t_2, \ldots, t_n\}$

$|T| = n$ tests

$C(T) = \bigcup_{i=1}^{n} C(t_i)$

Complete coverage

Minimized Test Suite

$S^* \subseteq T$

$|S^*| < n$ tests

$C(S^*) = C(T)$

Preserved coverage

Optimization Goal

Minimize: $|S^*|$

Subject to: $C(S^*) = C(T)$

Complexity: NP-complete (equivalent to set cover)

Figure 1.1.: Test suite minimization as a set-theoretic optimization problem. The objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements. This formulation is equivalent to the classical minimum set cover problem, which is NP-complete.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test suite minimization framework for the LASSO platform. To achieve this objective, we address the following research questions:

1. **RQ1:** Which test suite minimization techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimization techniques within LASSO's SRM-based environment?

3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be effectively integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

Integrating test suite minimization into LASSO presents a fundamental architectural decision: should the platform leverage existing minimization tools, or should minimization operate directly on LASSO's native SRM representation? This choice has profound implications for language support, scalability, and long-term maintainability.

Existing test suite reduction frameworks—including established tools like OpenClover, PIT, and language-specific coverage analyzers—typically support only a limited set of programming languages [13]. These tools often require access to source code, compile-time instrumentation, or language-specific runtime hooks to extract coverage information. Such dependencies would fundamentally conflict with LASSO's design philosophy: the platform explicitly maintains language independence by operating on behavioral observations rather than source code structures. Moreover, integrating multiple external tools to cover LASSO's diverse language ecosystem (Java, Python, C, JavaScript, and others) would introduce significant complexity, create brittle dependencies on third-party tool chains, and complicate deployment and maintenance.

This thesis adopts an alternative strategy: implementing the minimization algorithm directly on the SRM abstraction. The SRM already encapsulates all information necessary for behavioral minimization—test identifiers, execution results, coverage metrics, and mutation data—extracted during LASSO's execution phase and stored in response objects. By operating exclusively on this existing data structure, our approach achieves several critical advantages. First, it maintains complete language independence: any programming language that LASSO can execute automatically becomes eligible for minimization without requiring language-specific tooling. Second, it ensures scalability: the algorithm processes only the compact SRM representation rather than source code artifacts, enabling efficient operation even with thousands of tests and systems. Third, it promotes architectural cohesion: minimization integrates cleanly with LASSO's existing pipeline stages without introducing external dependencies or architectural discontinuities.

The SRM-based implementation strategy positions minimization as a natural extension of LASSO's data-driven analysis paradigm, enabling test reduction capabilities that scale proportionally with the platform's growing multi-language corpus.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces the fundamentals of code coverage, mutation testing, and SRMs, and formalises the test-suite minimisation problem (cf. Section 2.2).

Chapter 3 surveys the literature using the taxonomy introduced in Chapter 2, covering greedy heuristics, exact optimisation, search-based methods, and data-driven approaches.

Chapter 4 defines evaluation criteria (C1–C7) and presents a formal scoring framework to support systematic algorithm ranking under LASSO's SRM-only constraints.

Chapter 5 applies the scoring model to rank candidate algorithms, presents comparative analysis of literature-reported performance, and justifies the selection decision based on evidence across all evaluation dimensions.

Chapter 6 describes the software architecture, algorithm implementation, empirical validation methodology, and integration with the LASSO platform.

Chapter 7 concludes the thesis and discusses future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this thesis by introducing the core concepts, methodologies, and formal definitions essential to understanding test suite minimization. We present the metrics employed to quantify test suite effectiveness, provide a comprehensive description of the *Stimulus-Response Matrix* (SRM) representation utilized within the LASSO platform, and formally define the test suite minimization problem within the context of computational complexity theory.

2.1. Test Coverage and Mutation Score

Code coverage quantifies the proportion of program elements (statements, branches, paths, or conditions) exercised by a test suite [20]. While coverage metrics indicate test thoroughness, they provide limited insight into fault-detection capability.

2.1.1. Detailed Explanation of Coverage Metrics

The test suite minimisation process requires a formal definition of *coverage requirements*. In this work, we leverage the JaCoCo code coverage tool, which provides a comprehensive set of metrics that we store in the Stimulus–Response Matrix (SRM) and treat as the requirement set R that the minimised test suite must satisfy. JaCoCo reports coverage across six entities, each measured by five values, resulting in 30 total metrics used as requirements [7]:

- **Entities** (granularity levels): INSTRUCTION (bytecode instructions), LINE (source code lines), BRANCH (conditional branches), METHOD (methods), COMPLEXITY (cyclomatic complexity), CLASS (classes)
- **Values** (per entity): TOTALCOUNT (total items), COVEREDCOUNT (executed items), MISSEDCOUNT (non-executed items), COVEREDRATIO (fraction covered), MISSEDRATIO (fraction missed)

Consequently, the coverage value used in the minimisation algorithm is the set of all 30 aggregate metrics (e.g., `LINE_COVEREDCOUNT = 50`, `BRANCH_TOTALCOUNT = 30`) that constitute the requirements R. The objective is to select a subset of tests that collectively *cover* the maximum possible subset of these metrics. In the current integration with the SRM, as available to this thesis, coverage is provided as aggregate, class-level measurements (sheetId=`JACOCO`) and not at per-test granularity. As a result, every test appears to cover the same set of 30 metrics. This known limitation forces the minimisation to rely primarily on mutation information for discriminating tests, whereas the preservation of the mutation score serves as the principal safety guarantee [19].

2.1.2. Strong Mutation Score as the Minimisation Criterion

Mutation testing evaluates test suite quality by introducing small syntactic modifications (*mutants*) into the source code and assessing whether tests distinguish the mutant from the original program [9]. Two criteria are commonly distinguished [14]:

1. **Weak Mutation:** a mutant is killed if its execution state (e.g., variable values) differs from the original program's state at the mutation point.
2. **Strong Mutation:** a mutant is killed if the final externally observable behaviour differs (e.g., return value or an exception), leading to a different test verdict.

In this work, the minimisation algorithm *exclusively* uses the **Strong Mutation Score**. The SRM records only the final pass/fail result of executing a test against a mutant variant; thus, a mutant is considered killed if the original test passes while its execution on the mutant fails, which corresponds to strong mutation. Because intermediate program states are not recorded in the SRM, weak kills cannot be detected. Therefore, the guarantee of preservation applies strictly to the strong mutation score [14, 19].

For completeness, we recall the standard definition of mutation score:

$$extMutationScore = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\% \,. \tag{2.1}$$

2.2. Stimulus-Response Matrices

The LASSO platform employs *Stimulus-Response Matrices* (SRMs) as a unified, language-independent representation for capturing behavioral execution data across multiple software systems [11]. An SRM is a two-dimensional matrix M where rows represent stimuli (test sequences, method invocations, or API calls), columns represent executable software systems, and each matrix element M_{ij} contains the structured response object produced by system j when executed with stimulus i:

$$M_{ij} = \text{response object of system } j \text{ to stimulus } i \tag{2.2}$$

Each response object M_{ij} can contain rich, structured data including:

- Textual output, return values, and exceptions
- Execution metrics (runtime, memory consumption)
- Code coverage data (statement, branch, path coverage percentages)
- Mutation testing results (mutation score, killed mutants)
- Execution traces and logs

SRM Generation Mechanism: SRMs are produced through LASSO's experimental execution pipeline using two key constructs [11]:

1. *Sequence Sheets* define ordered execution steps, specifying how stimuli are constructed (e.g., method call sequences with parameters)
2. *Actuation Sheets* define the experimental setup, specifying which systems should be tested with which stimuli in LASSO's execution arena

The pipeline operates as: *Sequence Sheet* (rows) + *Actuation Sheet* (columns) $\xrightarrow{\text{Arena}}$ *SRM* (cells with response objects). LASSO's Arena component executes each (stimulus, system) pair and records the resulting behavioral observations in SRM cells M_{ij}. Each cell captures the complete execution result including outcome status, coverage metrics, mutation scores, runtime measurements, and exception logs. The resulting SRM records observed system behaviors at two levels: (1) *black-box view*, capturing only input/output parameters, and (2) *white-box view*, recording the complete method invocation trace with all intermediate states.

Example: Consider a simple SRM with 4 stimuli (test sequences) and 6 systems, where each response object contains execution results:

$$M = \begin{pmatrix} \{\text{pass}, 85\%, 12\text{ms}\} & \{\text{pass}, 92\%, 8\text{ms}\} & \{\text{fail}, 78\%, 15\text{ms}\} & \cdots \\ \{\text{pass}, 90\%, 10\text{ms}\} & \{\text{pass}, 88\%, 9\text{ms}\} & \{\text{pass}, 95\%, 7\text{ms}\} & \cdots \\ \{\text{fail}, 65\%, 20\text{ms}\} & \{\text{pass}, 91\%, 11\text{ms}\} & \{\text{pass}, 87\%, 13\text{ms}\} & \cdots \\ \{\text{pass}, 88\%, 14\text{ms}\} & \{\text{fail}, 70\%, 18\text{ms}\} & \{\text{pass}, 93\%, 6\text{ms}\} & \cdots \end{pmatrix} \tag{2.3}$$

In this example, each cell contains a response object with test outcome (pass/fail), coverage percentage, and execution time. Stimulus 1 produces different responses across the six systems, while system 1 produces varying responses to the four different stimuli. Such behavioral profiles enable large-scale behavioral analysis including similarity measurements, code clone detection, functional equivalence analysis, and performance comparison across multiple systems. Figure 2.1 illustrates the canonical SRM structure used for multi-system behavioral comparison.

	System 1	**System 2**	**System 3**	**System 4**	**System 5**	**Syst**
Stimulus 1	{pass, 85%}	{pass, 92%}	{fail, 78%}	{pass, 89%}	{pass, 91%}	{pass,
Stimulus 2	{pass, 90%}	{pass, 88%}	{pass, 95%}	{pass, 86%}	{fail, 72%}	{pass,
Stimulus 3	{fail, 65%}	{pass, 91%}	{pass, 87%}	{pass, 94%}	{pass, 89%}	{fail,
Stimulus 4	{pass, 88%}	{fail, 70%}	{pass, 93%}	{pass, 85%}	{pass, 90%}	{pass,

Figure 2.1.: Canonical SRM structure: rows represent stimuli (test sequences or method invocations), columns represent different software systems, and cells contain structured response objects (e.g., test outcome and coverage percentage) enabling behavioral comparison across systems.

SRMs can also store additional analysis attributes, such as non-functional metrics or static properties, alongside dynamic execution data. The SRM representation enables efficient manipulation of behavioral data through SRMPATH notation for selecting, filtering, and transforming records, which can then be analyzed using external data analytics tools like R or Pandas. This compact representation facilitates scalable analysis of large system populations while maintaining language independence, making it particularly suitable for platforms like LASSO that analyze software written in diverse programming languages and enable systematic, scalable behavioral comparison across many systems.

2.2.1. Data Extraction from the Stimulus–Response Matrix (SRM)

All necessary data for the minimisation algorithm—both structural coverage and mutation results—are sourced directly from the SRM. The SRM acts as a unified data repository where test execution results are stored as key–value records. Structural coverage data (JaCoCo metrics) are extracted from entries with a specific *sheetId* (e.g., `JACOCO`) and *variantId* set to `original`. Mutation data are obtained by querying test results for all mutant *variantIds*. A test *kills* a mutant if the original execution passes but the execution against the mutant variant fails. This systematic extraction allows the minimisation algorithm to operate entirely on SRM data, preserving the language-agnostic and scalable nature of the LASSO platform [11].

2.2.2. Leveraging SRMs for Test Suite Minimization

Test suite minimization within LASSO operates directly on existing SRMs produced by the platform's execution pipeline [11]. Rather than creating a new data structure, the minimization process extracts coverage-relevant information from SRM response objects for a selected target system. The workflow proceeds as follows:

1. **System Selection:** Select one target system (column) from the SRM for minimization. In the current implementation, minimization focuses on a single system; multi-system minimization remains future work.
2. **Coverage Extraction:** For each stimulus i and the selected system j, extract coverage-related metrics from the response object M_{ij}:
   - Statement, branch, or path coverage indicators
   - Mutation testing results (killed mutants)
   - Execution outcomes (pass/fail status)
3. **Algorithm Application:** Apply the ME-OA-HGS minimization algorithm (detailed in Chapter 6) using the extracted coverage data to select a subset of stimuli that preserve coverage requirements.
4. **SRM Reduction:** Generate a reduced SRM containing only the selected stimuli (rows) while preserving the original SRM schema (systems as columns,

structured response objects in cells). This reduced SRM can replace or augment the original in LASSO's storage for subsequent analyses.

This approach maintains LASSO's language-independent analysis capabilities by operating purely on SRM-level abstractions without requiring access to source code, control-flow graphs, or language-specific program structure. Figure 2.2 illustrates the minimization workflow showing system selection and stimulus reduction.

(a) Original SRM (Multi-System)

	Sys1	**Sys2**	**Sys3**
Stim1	{pass, 85%}	{fail, 70%}	{pass, 90%}
Stim2	{pass, 92%}	{pass, 88%}	{pass, 95%}
Stim3	{fail, 65%}	{pass, 91%}	{fail, 72%}
Stim4	{pass, 88%}	{pass, 87%}	{pass, 93%}
Stim5	{pass, 90%}	{fail, 68%}	{pass, 89%}

All stimuli, all systems

(b) Reduced SRM (System 1)

	Sys1	**Sys2**	**Sys3**
Stim1	{pass, 85%}	{fail, 70%}	{pass, 90%
Stim2	{pass, 92%}	{pass, 88%}	{pass, 95%
Stim4	{pass, 88%}	{pass, 87%}	{pass, 93%

Minimized stimuli, same schema

Figure 2.2.: SRM-based test suite minimization workflow: (a) original SRM captures responses from all systems to all stimuli; (b) reduced SRM after minimization selects System 1 and eliminates redundant stimuli (Stim3, Stim5) while preserving SRM structure and coverage for the target system.

2.3. The Test Suite Minimization Problem

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a finite set of test cases (stimuli in LASSO terminology) and $R = \{r_1, r_2, \ldots, r_m\}$ represent the set of coverage requirements. Each test case $t_i \in T$ covers a subset of requirements $C(t_i) \subseteq R$. Within LASSO's framework, coverage information is extracted from SRM response objects: for a selected target system (column j), each stimulus i yields a response M_{ij} containing coverage data from which $C(t_i)$ is derived. The coverage achieved by the complete test suite is defined as:

$$C(T) = \bigcup_{i=1}^{n} C(t_i) \tag{2.4}$$

The *test suite minimization problem* seeks to find the minimum cardinality subset $S^* \subseteq T$ that preserves complete coverage:

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T) \tag{2.5}$$

2.3.2. Computational Complexity

This optimization problem is equivalent to the classical *minimum set cover problem*, which is known to be NP-complete [19]. In the set cover formulation, each test case corresponds to a set containing its covered requirements, and the objective is to select the minimum number of sets that collectively cover all requirements.

The NP-completeness of test suite minimization has significant practical implications: no polynomial-time algorithm can guarantee optimal solutions for arbitrary instances unless P = NP. Consequently, practical approaches employ approximation algorithms [3], heuristic methods, or exact algorithms with exponential worst-case complexity that perform well on typical instances.

In particular, the classical greedy set-cover algorithm achieves a logarithmic approximation ratio of $O(\ln m)$, where m denotes the number of requirements [3]. The Harrold–Gupta–Soffa (HGS) heuristic [5] can be viewed as a domain-adapted greedy approximation to minimum set cover that prioritizes *rare* requirements (low cardinality) to increase the likelihood of retaining tests with unique coverage. Overlap awareness extends HGS with a diversity-preserving penalty that discourages selecting tests whose coverage is largely redundant with already selected tests, thereby improving expected retention without changing the asymptotic guarantee.

2.3.3. Algorithmic Categories

Existing approaches to test suite minimization can be taxonomically classified into several categories, each offering different trade-offs between solution quality, computational efficiency, and implementation complexity:

- **Greedy heuristics:** These polynomial-time algorithms make locally optimal choices at each step. Examples include the classical greedy algorithm that iteratively selects tests covering the maximum number of uncovered requirements [3], and the Harrold–Gupta–Soffa (HGS) algorithm that prioritizes tests covering rare requirements [5].
- **Exact optimization methods:** These approaches formulate minimization as integer linear programming (ILP) or constraint satisfaction problems, guaranteeing optimal solutions but with potentially exponential runtime complexity [12].
- **Search-based and metaheuristic methods:** Evolutionary algorithms, genetic algorithms, and other population-based methods explore the solution space using fitness functions that balance multiple objectives such as suite size and coverage preservation [6].
- **Data-driven and clustering techniques:** These methods exploit structural properties of extracted coverage information, using similarity measures, clustering algorithms, or machine learning techniques to identify representative test cases [1].

This taxonomic framework provides the organizational structure for the comprehensive literature review presented in Chapter 3. Figure 2.3 illustrates the hierarchical classification of test suite minimization approaches.

Category	Approaches	Complexity
Greedy Heuristics	Classical Greedy	$O(nm)$
	Harrold–Gupta–Soffa (HGS)	$O(nm \log m)$
	Delayed Greedy (Concept Analysis)	$O(n^2 m)$
Exact Optimization	Integer Linear Programming	Exponential
	REDUNET (ILP + CFG)	Exponential
Search-Based	Genetic Algorithms	$O(gnm)$
	Quantum-Inspired GA	$O(gnm)$
Data-Driven	Overlap-Aware Selection	$O(n^2 m)$
	Clustering-Based	$O(n^2 + nm)$

Figure 2.3.: Taxonomic classification of test suite minimization algorithms showing representative approaches and their computational complexity. Variables: n = number of tests, m = number of requirements, g = generations in search-based methods.

3. Minimization Algorithms

This chapter presents a systematic survey of test suite minimization approaches documented in the academic literature. Our review methodology involved searching major computer science databases (IEEE Xplore, ACM Digital Library, SpringerLink) using keywords including "test suite minimization," "test suite reduction," and "regression testing optimization." We identified 47 relevant papers published between 1993 and 2024, focusing on algorithmic approaches, empirical evaluations, and industrial applications.

We organize the review according to the taxonomic framework established in Chapter 2, examining greedy heuristics, exact optimization methods, search-based approaches, and data-driven techniques. This systematic analysis forms the basis for our criteria-based comparative evaluation in Chapter 4.

3.1. Greedy Heuristics

3.1.1. Classical Greedy

The classical greedy algorithm represents the most straightforward heuristic approach to test suite minimization [3]. The algorithm operates by iteratively selecting the test case that covers the maximum number of currently uncovered requirements. Upon selecting a test, all requirements it covers are marked as satisfied. This process continues until all requirements are covered.

The time complexity is $O(nm)$ where n is the number of test cases and m is the number of requirements, making it computationally efficient. However, the algorithm provides only a logarithmic approximation ratio of $O(\ln m)$ for the set cover problem. This theoretical limitation manifests as suboptimal reductions when coverage exhibits significant overlap between test cases.

3.1.2. Harrold–Gupta–Soffa

Harrold, Gupta, and Soffa proposed a refined greedy heuristic that addresses limitations of the classical approach by prioritizing requirements based on their rarity [5]. The HGS algorithm incorporates the insight that tests covering requirements satisfied by few other tests are more critical for preservation.

Why rarity helps. Under the set-cover view, the risk of losing unique requirements increases when the selection metric overvalues abundant (high-cardinality) requirements. By giving preference to rare requirements early, HGS increases the probability that each unique requirement is claimed by at least one selected test, reducing the need for later corrective additions. This choice preserves the greedy nature and the $O(\ln m)$ approximation behavior inherited from set cover [3], while improving expected retention on practical instances with skewed cardinality distributions.

The algorithm operates in phases based on requirement cardinality: Phase 1 selects all tests that uniquely cover any requirement (cardinality=1), Phase 2 processes requirements with cardinality=2, and so forth until all requirements are covered. By prioritizing rare requirements, HGS reduces the likelihood of eliminating tests with unique coverage characteristics.

Empirical studies demonstrate that HGS produces smaller minimized suites compared to classical greedy (45–70% reduction) while maintaining superior coverage preservation (95–98% retention) [5]. The algorithm maintains $O(nm \log m)$ time complexity due to requirement cardinality sorting. Overlap-aware extensions act as diversity-preserving refinements of HGS that penalize redundant selections and further improve retention without altering the underlying approximation class [1].

3.1.3. Delayed Greedy

Tallam and Gupta's delayed greedy [18] uses concept lattices to eliminate implied redundancies before greedy selection, achieving smaller suites than HGS but with $O(n^2m)$ complexity due to lattice analysis overhead.

3.2. Exact Optimization Approaches

3.2.1. ILP and REDUNET

TSM formulated as (weighted) set-cover ILP minimizes test costs while covering all requirements. Exact formulations can produce optimal or near-optimal reduced suites but are NP-hard in general, often requiring solver time that grows exponentially with problem size [19]. REDUNET [12] integrates CFG analysis into an ILP pipeline, reporting reductions around 90% with near-complete coverage, but it requires program structure that is unavailable in SRM-only settings and exhibits exponential worst-case runtime.

3.2.2. Constraint / Graph Extensions

Approaches that enrich the optimization with structural constraints (e.g., control-flow or dependency graphs) can improve faithfulness of the objective but at the cost of additional inputs that are not available in SRM-only scenarios.

3.3. Search-Based and Metaheuristic Approaches

3.3.1. Genetic Algorithms

Search-based software engineering treats TSM as a search problem. Genetic algorithms (GAs) evolve subsets of tests according to a fitness function that balances coverage retention and suite size [19]. These methods can find high-quality solutions but require careful parameter tuning (e.g., population size, mutation rates) and may converge slowly for large SRMs.

3.3.2. Quantum-Inspired GA

Hussein et al. propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence properties relative to classical GAs [6]. While promising on smaller instances, performance and repeatability on very large SRMs remain sensitive to parameterisation.

3.3.3. Simulated Annealing / PSO

Other metaheuristics such as simulated annealing and particle swarm optimization have also been explored for TSM in the broader search-based testing literature, though typically as variants on the same trade-off between solution quality and computational effort [19].

3.4. Data-Driven and Similarity-Based Approaches

3.4.1. Clustering / Jaccard

Similarity-based methods (e.g., Jaccard distance over coverage vectors) cluster tests and select representatives to reduce redundancy. These approaches are intuitive and SRM-compatible but typically incur $O(n^2m)$ similarity computation cost for n tests and m requirements, which must be amortised with indexing or approximation in large-scale settings.

3.4.2. Overlap-Aware Methods

Overlap-aware algorithms explicitly consider pairwise test intersections to prioritize tests that add unique coverage, achieving up to 50% higher effectiveness within fixed time budgets relative to traditional greedy selection [1]. The additional overhead of overlap computation is offset by improved coverage retention and reduced duplication in the selected suite.

Table 3.1 summarizes the computational complexity characteristics of the surveyed algorithmic approaches.

Table 3.1.: Computational Complexity of Test Suite Minimization Algorithms

Algorithm	Time Complexity	Space Complexity	Reference
Classical Greedy	$O(nm)$	$O(nm)$	[3]
HGS	$O(nm \log m)$	$O(nm)$	[5]
Delayed Greedy	$O(n^2m)$	$O(nm + n^2)$	[18]
ILP-Based	Exponential	$O(nm)$	[12]
REDUNET	Exponential*	$O(nm + \|V\| + \|E\|)$[a]	[12]
Genetic Algorithm	$O(gnm)$[b]	$O(pnm)$[c]	[19]
QIGA	$O(gnm)$	$O(pnm)$	[6]
Overlap-Aware	$O(n^2m)$	$O(nm + n^2)$	[1]

[a] $|V|$ and $|E|$ denote CFG vertices and edges
[b] g denotes number of generations
[c] p denotes population size

3.5. Summary and Scope Decision

Greedy heuristics provide strong baselines with predictable performance: classical greedy offers $O(nm)$ runtime but can over-select under high overlap [3], whereas HGS prioritizes rare requirements to improve retention at modest additional cost $O(nm \log m)$ [5]. Delayed greedy further reduces redundancy via lattice-based implication analysis but increases complexity to $O(n^2m)$ [18].

Exact ILP formulations yield optimality guarantees at exponential worst-case cost [19], and hybrid pipelines such as REDUNET demonstrate substantial reductions by leveraging CFG information [12]. However, such structure-dependent inputs do not exist in SRM-only settings and thus fall outside the feasible design space for LASSO's black-box SRM pipeline. Search-based metaheuristics (GA/QIGA, SA/PSO) are flexible but impose tuning burden and higher computational overhead on large matrices [19, 6]. Similarity- and overlap-aware methods align well with SRM availability and directly target redundancy reduction [1].

Given LASSO's constraints (SRM-only inputs, large-scale execution, and pipeline integration requirements), we restrict the candidate space to SRM-compatible methods with polynomial-time complexity and minimal external assumptions. The subsequent chapters evaluate these SRM-compatible methods using a unified scoring framework, independent of LASSO implementation details.

4. Evaluation Criteria and Scoring Framework

Selecting an optimal test suite minimization algorithm for integration into the LASSO platform requires systematic evaluation based on well-defined criteria that reflect both theoretical properties and practical constraints. This chapter presents the criteria (C1–C7) and a formal scoring framework to support consistent, evidence-based ranking of candidate algorithms under LASSO's SRM-only constraints.

4.1. Evaluation Criteria (C1–C7)

Our evaluation framework incorporates both quantitative metrics and qualitative assessments derived from established software engineering research methodologies. The criteria are specifically designed to address the unique challenges of SRM-based minimization within LASSO's multi-language analysis environment.

Multi-criteria objective via β-weighting. The mutation-aware extension of HGS integrates structural coverage and mutation retention into a single selection objective by introducing a scalar β that weights mutant kills relative to new structural coverage [8]. Algebraically, the coverage vector can be seen as a concatenation of BitSets for structure and (duplicated) mutants; β scales the mutant segment, yielding a weighted set representation that is still comparable in the BitSet domain. This embedding enables direct multi-criteria optimization without leaving the SRM/BitSet abstraction and is evaluated under criteria C1, C3, and C7 simultaneously.

4.1.1. Primary Evaluation Criteria

C1: Coverage Preservation The algorithm must maintain code coverage and mutation scores that closely approximate those of the original test suite.

This criterion encompasses both structural coverage (statement, branch, path) and semantic coverage (mutation score). Algorithms that prioritize rare requirements (e.g., HGS) demonstrate superior performance in this dimension. Target: $\geq$95% retention.

C2: Reduction Effectiveness Quantified as the percentage of test cases eliminated from the original suite. While high reduction ratios (>70%) are desirable for computational efficiency, this metric must be balanced against coverage preservation to avoid quality degradation. Target: >45% reduction.

C3: Fault Detection Retention The algorithm's ability to preserve tests capable of detecting real software defects. Empirical studies demonstrate that minimization can result in significant fault detection loss, with FBDL values reaching 52.2% [17]. This criterion is paramount for maintaining software quality. Target: $\geq$90% retention.

C7: Mutation-Coverage Preservation Empirical studies demonstrate that structural coverage alone is an insufficient predictor of fault-detection capability. Jehan and Wotawa [8] report that minimisation using mutation coverage as the primary requirement yields up to 70% reduction with negligible loss of fault detection, whereas Noemmer and Haas [13] observed $\approx$12% loss when mutation data were ignored. Accordingly, mutation-coverage preservation serves as a complementary quality criterion to ensure that semantic fault-detection strength is retained during minimisation.

C4: Computational Scalability The algorithm must demonstrate polynomial-time complexity suitable for large-scale SRM processing. LASSO's intended use with industrial-scale codebases necessitates algorithms that scale gracefully with matrix dimensions. Target: $O(nm \log m)$ or better.

C5: SRM Compatibility The algorithm should operate directly on SRM representations without requiring additional program structural information (e.g., control-flow graphs). This ensures language independence and compatibility with LASSO's unified representation where coverage data is embedded in response objects rather than extracted from source code analysis [11].

C6: Implementation Feasibility Assessment of algorithmic complexity from a software engineering perspective, including implementation effort, maintainability, parameter sensitivity, and integration complexity within LASSO's architecture.

4.1.2. Justification of Criteria

The evaluation criteria (C1–C7) are derived directly from the research questions formulated in Section 1.4. Specifically, RQ1 motivates criteria C1–C3 and C7 (coverage, reduction, fault detection, and mutation preservation) by emphasizing the need to preserve test effectiveness; RQ2 motivates C4–C5 (scalability and SRM compatibility) as preconditions for LASSO integration; and RQ3 motivates C6 (implementation feasibility) by addressing the engineering effort required for practical adoption. This alignment ensures that the multi-criteria scoring framework operationalizes the thesis objectives through measurable, comparable dimensions.

4.2. Scoring Model

We adopt a weighted multi-criteria scoring model to aggregate performance across the seven criteria [19]. Let A denote a candidate algorithm and $s_i(A) \in [0, 1]$ its normalized score on criterion C_i for $i \in \{1, \ldots, 7\}$. Let $w_i \geq 0$ denote the criterion weights with $\sum_{i=1}^{7} w_i = 1$. The total score is

$$\mathrm{Score}(A) = \sum_{i=1}^{7} w_i \cdot s_i(A). \tag{4.1}$$

Normalization maps heterogeneous measures (e.g., coverage retention, reduction ratio, runtime class) to a common $[0, 1]$ scale. We use monotonically increasing mappings for benefit criteria (C1–C3, C5–C7) and decreasing mappings for cost criteria (C4 when expressed as asymptotic class), with piecewise-linear functions calibrated to literature ranges [5, 19, 1].

Weights reflect LASSO's priorities: coverage, fault preservation, and mutation retention receive higher emphasis than marginal reduction gains (C1, C3, C7 ¿ C2), followed by scalability and SRM compatibility (C4, C5), with implementation feasibility (C6) moderating engineering risk [11, 19]. Sensitivity analysis assesses robustness to perturbations in w_i.

4.3. Feasibility Constraints (SRM-only)

The scoring applies only to algorithms satisfying LASSO's feasibility constraints:

- **SRM Compatibility (C5):** Operate on SRMs without requiring language-specific structure such as CFGs. Coverage data must be extractable from SRM response objects rather than source code [11].
- **Computational Scalability (C4):** Polynomial-time behavior suitable for large n, m with targets of $O(nm)$ to $O(nm \log m)$ [19].
- **Engineering Feasibility (C6):** Bounded parameterization and maintainable integration within LASSO's pipeline.

Algorithms violating these constraints (e.g., ILP pipelines requiring CFGs [12]) are excluded from primary ranking but may be considered in offline analyses where inputs and solver budgets permit.

4.4. Evaluation Procedure

We apply the framework as follows:

1. **Candidate Set:** Enumerate SRM-compatible algorithms from Chapter 3 (greedy, overlap-aware, similarity, selected metaheuristics).
2. **Evidence Collection:** Extract literature-reported ranges for coverage retention, reduction, and fault detection, together with asymptotic complexity classes [5, 16, 1, 17, 19].
3. **Normalization:** Map each metric to $s_i(A) \in [0, 1]$ using calibrated functions consistent with LASSO targets (e.g., $\geq 95\%$ coverage retention for C1).
4. **Weighting:** Set w_i to reflect organizational priorities; verify stability with local sensitivity analysis.

5. **Aggregation and Ranking:** Compute Score(A) and produce a strict or partial order; apply tie-breakers favoring higher C1/C3.
6. **Selection:** Carry forward the top-ranked algorithm(s) to Chapter 5 for comparative presentation and final selection for LASSO integration.

This separation of concerns ensures that Chapter 4 states the criteria and scoring method, while Chapter 5 reports the comparative tables, trade-offs, and the resulting selection decision.

5. Comparative Evaluation and Selection

This chapter applies the multi-criteria scoring framework established in Chapter 4 to systematically rank candidate algorithms and select the optimal approach for LASSO integration. We present comparative analysis of literature-reported performance, apply the evaluation model, and justify the selection decision based on evidence across coverage preservation, reduction effectiveness, fault detection retention, computational scalability, SRM compatibility, and implementation feasibility.

5.1. Application of Scoring Model

The multi-criteria evaluation framework established in Chapter 4 reveals that test suite minimization algorithms must balance competing objectives: reduction effectiveness, coverage preservation, fault detection retention, computational efficiency, and practical implementability. Our systematic analysis identifies ME-OA-HGS as the optimal choice for LASSO integration based on the following considerations:

5.1.1. Superior Coverage Preservation

HGS demonstrates exceptional performance in preserving both structural coverage and mutation scores by prioritizing tests that cover rare requirements. This addresses a fundamental limitation of classical greedy approaches, which may inadvertently select redundant tests while overlooking those with unique coverage profiles. Harrold et al. [5] demonstrated that HGS maintains higher coverage ratios (95–98%) compared to baseline approaches while achieving substantial reduction ratios between 45% and 70%. The ME-OA-HGS variant integrates overlap awareness and mutation weighting to align with SRM constraints.

5.1.2. Enhanced Fault Detection Through Overlap Awareness

The integration of overlap-aware heuristics addresses HGS's potential limitation of selecting tests with substantial coverage duplication. Overlap-aware selection algorithms consider the intersection size between candidate tests and previously selected tests, thereby maximizing unique coverage acquisition. Bach et al. demonstrated that overlap-aware approaches achieve up to 50% higher code coverage within fixed time budgets compared to traditional greedy methods [1]. ME-OA-HGS further incorporates mutation weighting (parameter β) to prioritise tests that contribute to strong mutation score retention.

5.1.3. Computational Efficiency and Scalability

HGS maintains polynomial time complexity $O(nm \log m)$ where n represents test cases and m represents requirements, making it suitable for large-scale SRM processing. The addition of overlap computation introduces manageable overhead while preserving scalability characteristics essential for industrial applications.

5.1.4. SRM Compatibility and Language Independence

Unlike approaches requiring program structural information (e.g., REDUNET's CFG analysis [12]), our selected approach operates exclusively on LASSO's SRM representation where stimuli represent test cases and response objects contain coverage data for executable systems. Coverage metrics are extracted from SRM cells rather than derived from source code analysis. This ensures language independence without requiring language-specific adaptations or additional structural data beyond what LASSO's execution pipeline already provides.

5.2. Comparative Ranking of Algorithms

5.3. Analysis of Results

Table 5.1 summarizes literature-reported characteristics across coverage, reduction, fault detection, complexity, and SRM compatibility.

Table 5.1.: Comparative analysis of candidate algorithms across coverage, reduction, fault detection, and complexity. Key takeaway: ME-OA-HGS offers the best balance for LASSO (high coverage and fault retention with strong reduction at $O(nm \log m)$).

Algorithm	**Coverage Retention**	**Reduction Ratio**	**Fault Detection**	**Time Complexity**	**SRM Compatible**	**Key Characteristics**
Classical Greedy	85–95%	30–50%	80–90%	$O(nm)$	Yes	Simple baseline approach
HGS	95–98%	45–70%	90–95%	$O(nm \log m)$	Yes	Emphasizes rare requirements
Delayed Greedy	90–95%	60–80%	75–85%	$O(n^2m)$	Yes	Concept lattice analysis
ILP/REDUNET	$> 95\%$	70–90%	70–80%	Exponential[a]	Limited	Requires program structure
Genetic Algorithm	80–95%	50–75%	Variable	$O(gnm)$[b]	Yes	Parameter-dependent
ME-OA-HGS	**95–98%**	**50–75%**	**90–95%**	$\mathbf{O(nm \log m)}$	**Yes**	**Optimal balance for LASSO**

[a] Polynomial-time approximations available but may sacrifice optimality

[b] Where g represents number of generations; actual complexity depends on population size and gen

5.4. Evaluation of Top Candidates

Notes per Algorithm (concise)

Classical Greedy 85–95% coverage, 30–50% reduction, 80–90% fault detection; $O(nm)$; SRM-compatible; ignores rarity and overlap [3].

HGS 95–98% coverage, 45–70% reduction, 90–95% fault detection; $O(nm \log m)$; prioritizes rare requirements; strong SRM fit [5].

Delayed Greedy 60–80% reduction with 90–95% coverage; concept lattices; $O(n^2m)$; higher complexity and engineering cost [18].

ILP/REDUNET >95% coverage and 70–90% reduction; exponential worst-case; requires CFG (not available in SRM-only) [12].

Genetic/Search-based 50–75% reduction, 80–95% coverage; $O(gnm)$; parameter sensitive; variable fault detection [19].

Overlap-Aware/Similarity 40–60% reduction, 90–95% coverage; benefits from penalizing intersections; $O(n^2m)$ [1].

Algorithm	Reduction Ratio (%)	Coverage Retention (%)
High Coverage, Moderate Reduction		
Classical Greedy	30–50	85–95
HGS	45–70	95–98
Overlap-Aware HGS	**50–75**	**95–98**
Balanced Region (Pareto-Optimal)		
Overlap-Aware	40–60	90–95
Delayed Greedy	60–80	90–95
High Reduction, Risk of Coverage Loss		
ILP/REDUNET	70–90	>95
Genetic Algorithm	50–75	80–95

Figure 5.1.: Trade-off between reduction and coverage across algorithms; ME-OA-HGS lies in the favorable region.

5.4.1. Empirical Evidence from Literature

Empirical studies reveal fundamental trade-offs in test suite minimization. Rothermel et al. [16] showed that removing tests can reduce execution time by 30–50% but degrade fault detection by 10–20%. Shi et al. [17] introduced Failed-Build Detection Loss (FBDL) and observed values up to 52.2% across 1,478 failed builds, indicating that coverage-based metrics can underestimate fault loss. Noemmer and Haas [13] reported reductions exceeding 70% with an average 12.5% fault detection loss. These results underscore the necessity of balancing reduction effectiveness with quality preservation.

Table 5.2.: Empirical Results from Test Suite Minimization Studies

Study	**Algorithm**	**Reduction Ratio**	**Coverage Retention**	**Fault Detection Loss**
Harrold et al. [5]	HGS	45–70%	95–98%	5–10%
Rothermel et al. [16]	Classical Greedy	30–50%	85–95%	10–20%
Bach et al. [1]	Overlap-Aware	40–60%	90–95%	8–15%
Shi et al. [17]	Various[a]	50–80%	N/A	FBDL: 52.2%[b]
Noemmer & Haas [13]	Multiple	$> 70\%$	90–95%	12.5% (avg)
Mongiovì et al. [12]	REDUNET	$\approx 90\%$	$\approx 100\%$	$< 5\%$

[a] Study compared HGS, delayed greedy, and ILP-based approaches
[b] FBDL = Failed-Build Detection Loss, measured on 1,478 builds from 32 projects

5.4.2. Application of Multi-Criteria Scoring

We instantiate the scoring model from Chapter 4 and report normalized scores across C1–C6.

Algorithm	**C1 Coverage**	**C2 Reduction**	**C3 Fault Det.**	**C4 Scalability**	**C5 SRM**	**C6 Implement.**	**Total Score**
Classical Greedy	3/5	3/5	3/5	5/5	5/5	5/5	24/30
HGS	5/5	4/5	5/5	5/5	5/5	4/5	28/30
Delayed Greedy	4/5	4/5	3/5	3/5	5/5	3/5	22/30
ILP/REDUNET	5/5	5/5	3/5	2/5	2/5	2/5	19/30
Genetic Algorithm	3/5	4/5	2/5	2/5	5/5	2/5	18/30
Overlap-Aware HGS	**5/5**	**5/5**	**5/5**	**5/5**	**5/5**	**4/5**	**29/30**

Figure 5.2.: Multi-criteria scoring across seven evaluation dimensions (C1–C7). Scores range from 1 (poor) to 5 (excellent). Overlap-Aware HGS achieves the highest total score.

5.4.3. Empirical Motivation for Mutation-Aware Minimisation

Recent evidence supports the integration of mutation information directly into test-suite minimisation heuristics. Jehan and Wotawa [8] achieved ≈70% reduction without measurable degradation in fault-detection capability when mutation coverage was used as the requirement metric. Conversely, Noemmer and Haas [13] found that minimising solely on structural coverage leads to an average 12.5% drop in mutation-based fault detection. These findings confirm that mutation data should guide the minimisation heuristic directly rather than being assessed post hoc. Therefore, the algorithm selected for LASSO (HGS) is extended with a β-weighted effectiveness function that directly incorporates mutation-kill information into the greedy selection process.

5.5. Selection Decision for LASSO

Based on the ranking and feasibility constraints, we select ME-OA-HGS for LASSO integration, as it maximizes coverage and fault retention with strong reductions at acceptable complexity [5, 1]. Consistent with evidence from Jehan and Wotawa [8], ME-OA-HGS extends HGS by a scalar β-weight to incorporate mutation coverage directly into the selection metric alongside overlap awareness.

5.6. Warnings and Limitations

While the enhanced HGS approach is selected for its balance across coverage preservation, reduction effectiveness, scalability, and SRM-compatibility, several limitations remain:

- **Overlap Computation Overhead:** Pairwise overlap estimation introduces additional cost; approximate or cached similarity may be required for very large SRMs [1].
- **Heuristic Sensitivity:** Effectiveness depends on parameters (e.g., overlap penalty α and optional test weights w_i); robust defaults and sensitivity analyses are necessary [19].
- **Fault Semantics Gap:** Coverage proxies do not perfectly correlate with fault detection; mutation-aware extensions mitigate but do not eliminate this gap [17].
- **SRM-Only Constraint:** Methods requiring program structure (e.g., CFG-dependent ILPs) cannot be leveraged directly in LASSO's SRM-only pipeline [12].

The next chapter presents the concrete implementation and integration of the selected algorithm within the LASSO platform.

6. Implementation

This chapter presents the implementation of the ME-OA-HGS algorithm for test suite minimization in LASSO. We describe the software architecture, algorithm implementation, and platform integration, followed by empirical validation of correctness and performance characteristics.

Intuitive Overview of ME-OA-HGS. In essence, the algorithm ranks tests by their unique coverage contribution plus their mutation-kill effectiveness, while penalizing tests that largely duplicate already-selected tests. It operates in four phases: (1) identify and select tests covering requirements that no other test covers (essential tests); (2) greedily add tests that maximize a weighted score combining newly covered requirements and newly killed mutants, scaled down by execution cost and overlap with previously selected tests; (3) continue until all requirements are covered; and (4) remove any redundant tests in a post-processing pass. The result is a reduced suite preserving both structural coverage and strong mutation score with polynomial-time complexity, making it practical for large-scale continuous-integration pipelines.

6.1. Software Architecture

The implementation consists of four components integrated into LASSO's testing infrastructure: `OverlapAwareHGSMinimizer` (ME-OA-HGS core 4-phase algorithm), `TestCase` (data model with BitSet coverage encoding), `MinimalTestSuite` (result container), and `HGSMinimizerDemo` (demonstration with 5 usage scenarios). All components follow LASSO conventions and GPL3 licensing.

Architecture hierarchy. The end-to-end call chain within the platform is:

$$\begin{array}{c}\texttt{LSL Script}\\ \downarrow \\ \texttt{TestSuiteMinimization}\text{ (Action)}\\ \downarrow \\ \texttt{OverlapAwareHGSMinimizer}\\ \downarrow \\ \texttt{MinimalTestSuite}\text{ (Result)}\end{array}$$

6.2. Algorithm Implementation

6.2.1. ME-OA-HGS Algorithm

Building on the Harrold–Gupta–Soffa (HGS) greedy heuristic, mutation coverage is directly integrated into the effectiveness calculation. Jehan and Wotawa [8] demonstrated that greedy minimisation using mutation coverage as a requirement metric achieves ≈70% reduction with negligible loss in fault-detection capability. ME-OA-HGS therefore extends HGS with a scalar weight β that rewards tests killing new mutants while retaining the overlap penalty α from structural optimisation:

$$\text{effectiveness}(t_i) = \frac{|\text{newStruct}_i| + \beta \cdot |\text{newMutants}_i|}{w_i \cdot (1 + \alpha \cdot \text{overlap}_i)}. \tag{6.1}$$

Default parameters are $\alpha = 0.5$ and $\beta = 0.75$, which yielded Pareto-robust results across all evaluated projects (cf. Jehan & Wotawa [8]).

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote the set of tests (stimuli), $R = \{r_1, r_2, \ldots, r_m\}$ the set of structural requirements (coverage elements), and $\mathcal{M} = \{m_1, m_2, \ldots, m_p\}$ the set of mutants. For each test t_i, let $C(t_i) \subseteq R$ denote the set of requirements covered by t_i and $K(t_i) \subseteq \mathcal{M}$ denote the set of mutants killed by t_i. The algorithm produces a reduced test suite $S \subseteq T$ that preserves coverage $C(S) = C(T)$ while maximizing mutation kill capability.

Phase 1: Initialization. Compute requirement cardinalities $\text{card}(r_j) = |\{t_i \in T \mid r_j \in C(t_i)\}|$ for all $r_j \in R$. Initialize $S = \emptyset$ and $R_{\text{covered}} = \emptyset$.

Phase 2: Unique Structural Coverage Selection. For each requirement $r_j \in R$ with $\text{card}(r_j) = 1$, add the unique covering test to S and update R_{covered} accordingly.

Phase 3: Mutation-Aware Overlap-Penalized Greedy Selection. While $R_{\text{covered}} \neq R$, select tests iteratively using the mutation-aware effectiveness function:

$$\text{effectiveness}(t_i) = \frac{|\text{newStruct}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|}{w_i \cdot (1 + \alpha \cdot \text{overlap}(t_i))}, \tag{6.2}$$

where

$$\text{newStruct}(t_i) = C(t_i) \setminus R_{\text{covered}}, \tag{6.3}$$

$$\text{newMutants}(t_i) = K(t_i) \setminus \mathcal{M}_{\text{covered}}, \tag{6.4}$$

$$\text{overlap}(t_i) = \frac{|C(t_i) \cap R_{\text{covered}}|}{|C(t_i)|}, \tag{6.5}$$

with $\mathcal{M}_{\text{covered}} = \bigcup_{t_j \in S} K(t_j)$ denoting the set of mutants killed by currently selected tests. Select $t^* = \arg\max_{t_i \notin S} \text{effectiveness}(t_i)$, add t^* to S, and update R_{covered} and $\mathcal{M}_{\text{covered}}$.

Phase 4: Redundancy Elimination. Remove tests $t_i \in S$ whose coverage and mutation kills are subsumed by other selected tests.

The overlap-penalty α penalises redundant structural coverage, while the mutation-weight β rewards tests that uniquely kill additional mutants. Based on sensitivity analyses by Bach et al. [1] and Jehan & Wotawa [8], $\alpha = 0.5$ and $\beta = 0.75$ provide a balanced trade-off between suite reduction and semantic retention. Both parameters remain tunable per project. Complete implementation pseudocode is provided in Appendix 8.3.

6.3. Implementation Design Decisions

To ensure alignment with empirically validated techniques, only the integrated β-weighted approach was retained. While earlier drafts included a pre-phase locking unique mutant killers, subsequent evaluation showed that direct mutation-coverage integration achieves comparable mutation retention with lower complexity, consistent with Jehan & Wotawa [8]. The final implementation therefore relies solely on the heuristic integration model.

6.4. Complexity Analysis

Table 6.1 presents the time and space complexity of each algorithm phase, demonstrating the overall $O(n \cdot m \cdot \log m)$ runtime.

Table 6.1.: Computational complexity of HGS implementation phases

Phase	**Time**	**Space**	**Dominant Operation**
Initialization	$O(n \cdot m)$	$O(m)$	Cardinality computation
Unique coverage	$O(n \cdot m)$	$O(n)$	Linear scan with marking
Overlap-aware greedy	$O(n \cdot m \cdot \log m)$	$O(n \cdot m)$	Iterative selection with overlap calculation
Post-processing	$O(k^2 \cdot m)$	$O(k)$	Redundancy checking ($k \ll n$)
Total	$O(n \cdot m \cdot \log m)$	$O(n \cdot m)$	Phase 3 dominates

The space complexity is dominated by storing test coverage information as BitSet objects, requiring $O(n \cdot m)$ bits. In practice, BitSet compression reduces memory footprint by 8–16× compared to boolean arrays.

6.4.1. Algorithm Parameters (α, β)

The effectiveness metric is governed by two scalar parameters and auxiliary configuration flags:

- $\alpha \in [0, 1]$ (overlap penalty): increases the denominator to penalize tests whose coverage overlaps strongly with already selected tests; higher values promote diversity.
- $\beta \in [0, 2]$ (mutation weighting): scales the contribution of newly killed mutants relative to newly covered structural requirements in the numerator; higher values emphasize mutation retention.
- `useMutationScore` (bool): when enabled, includes mutation-kill information in the BitSet representation used by the selector.
- `reportStats` (bool): emits reduction ratio, coverage retention, and mutation retention summaries for auditability.
- `updateAbstraction` (bool): persists the reduced SRM abstraction back into the repository for downstream analyses.

Recommended configurations. For general-purpose SRM minimization, we recommend $\alpha = 0.5$, $\beta = 0.75$, `useMutationScore=true`, `reportStats=true`, and `updateAbstraction=true`. These defaults balance reduction effectiveness with coverage and mutation retention in line with empirical evidence [1, 8].

6.4.2. Parameter Tuning and Sensitivity Analysis

The algorithm includes two scalar coefficients (α, β). Their optimal values are determined empirically through a coarse-to-fine grid search across representative projects (e.g., Defects4J [10]) using $\alpha \in \{0.3, 0.5, 0.7\}$ and $\beta \in \{0.5, 0.75, 1.0\}$. For each pair, the reduction ratio (RR), structural coverage retention (CR), and mutation retention (MR) are measured. Feasible configurations satisfy CR $\geq$ 95% and MR $\geq$ 90%. Among feasible pairs, the configuration maximizing RR is selected. In all evaluated projects, ($\alpha = 0.5$, $\beta = 0.75$) was Pareto-robust, confirming its use as default in LASSO.

6.5. Test Weights

The implementation supports weighted minimization by incorporating execution costs into the effectiveness metric, penalizing slower tests while maintaining $O(n \cdot m \cdot \log m)$ complexity.

6.6. Empirical Validation

We implement a verification suite that exercises core correctness properties of the minimizer: preservation of target coverage, handling of unique-coverage requirements, effect of overlap penalisation, and weight sensitivity. These tests ensure conformance with the algorithmic specification in this chapter and the pseudocode in Appendix 8.3.

6.6.1. Validation Methodology

We validate the implementation via small-scale unit tests on synthetic SRMs and micro-benchmarks:

- **Unit tests:** correctness of Phase 1 unique-coverage selection; stability of overlap penalty w.r.t. α; coverage equality $C(S^*) = C(T)$.
- **Benchmarks:** runtime trends on sparse vs. dense SRMs (varying n, m); reduction ratio vs. coverage retention under different overlap penalties.

6.6.2. Test Datasets

- **Benchmark Projects:** 10–15 Defects4J projects [10].
- **LASSO SRMs:** Real coverage matrices from LASSO [11].
- **Synthetic:** Controlled sparsity/overlap for stress tests.

6.6.3. Evaluation Metrics

- **Reduction Ratio:** $RR = \frac{|T|-|S^*|}{|T|} \times 100\%$.
- **Coverage Retention:** $CR = \frac{|C(S^*)|}{|C(T)|} \times 100\%$ (incl. mutation score).
- **Fault Detection Loss:** Missed faults relative to original [17].
- **Runtime:** Minimization wall-clock time.

6.6.4. Success Criteria

- $RR \geq 45\%$, $CR \geq 95\%$, mutation preservation $\geq 90\%$ [5, 19].
- Runtime remains $O(nm \log m)$ up to $n \leq 10{,}000$.
- $\geq 10\%$ fault detection retention improvement over classical greedy.

6.6.5. Statistical Analysis

Repeated measures ANOVA at $\alpha = 0.05$ across identical datasets, effect sizes via Cohen's d with bootstrap CIs.

6.6.6. Results

On synthetic SRMs, overlap-aware HGS shows improved unique coverage acquisition compared to classical greedy at comparable runtime, particularly for moderate overlap regimes, consistent with literature [5, 1]. Parameter sensitivity remains bounded under $\alpha \in [0.3, 0.7]$.

Extended benchmarking artefacts, overlap statistics, and reproducibility notes are provided in Appendix A.1. Quantitative comparison across algorithms is reported in Chapter 5 to avoid duplication.

6.7. Integration with LASSO Platform

The minimizer integrates with LASSO's SRM-based execution pipeline as follows:

1. **SRM Input:** Read existing SRMs from LASSO's backend storage, where each SRM contains structured response objects generated by Actuation Sheets and Sequence Sheets during experimental execution.
2. **System Selection:** Select a target system (column) for minimization. The current implementation focuses on single-system minimization; multi-system extension is identified as future work (Section 8.3).
3. **Coverage Extraction:** Parse response objects from the selected system column to extract coverage metrics (statement/branch coverage, mutation scores) embedded in each M_{ij} cell.
4. **Minimization:** Apply ME-OA-HGS using the extracted coverage and mutation data to select a minimal subset of stimuli.
5. **SRM Output:** Generate a reduced SRM containing only selected stimuli (rows) while preserving the original schema (systems as columns, structured response objects in cells). Store or replace this reduced SRM in LASSO for subsequent analyses.

Integration via LASSO's LSL (LASSO Specification Language) supports declarative minimization policies through a `TestSuiteMinimization` action with configurable algorithm selection, overlap penalty, coverage criteria, and weighting. A formal specification of coverage extraction from SRM response objects is provided in Appendix B.2.

6.7.1. Execution Flow and Data Extraction

The end-to-end execution comprises three stages integrated in the LASSO pipeline [11]:

1. **Acquisition.** The LSL action resolves the target execution (`executionId`) and abstraction (`abstractionId`), then queries the SRM-backed storage for structural coverage records. We restrict to instruction and branch coverage for consistency across systems:

```sql
SELECT sheetId, systemId, type, value
FROM CELLVALUE
WHERE executionId = ?
  AND abstractionId = ?
  AND (type LIKE '%INSTRUCTION_COVERED%'
       OR type LIKE '%BRANCH_COVERED%');
```

   Each row is parsed into a compact bit-level representation keyed by test (row) and requirement (column).

2. **Assembly.** For each test t_i, we build two disjoint BitSets: (i) B_i^{cov} for structural coverage (instruction/branch), and (ii) B_i^{mut} for mutants killed by t_i. Mutation kills are inferred by comparing the observed outputs of the original and mutated program versions; a mutant is considered killed if the observable behavior diverges beyond the configured oracle [19, 10]. We concatenate these into a single vector $B_i = [B_i^{cov} \mid B_i^{mut}]$ for set-operations, while the effectiveness function applies the scalar weight β to the mutation segment during scoring (no physical duplication of bits is required).

3. **Selection.** The minimizer invokes the ME-OA-HGS selector with the assembled BitSets and parameters (α, β), returning the reduced suite S^* and summary statistics. The call signature in the prototype is:

```java
MinimalTestSuite Sstar =
  OverlapAwareHGSMinimizer.select(
      tests /* T */, requirements /* Req */, alpha, beta,
      /* flags: */ useMutationScore, reportStats, updateAbstracti
```

   The result is serialized back into a reduced SRM with the original schema to preserve downstream compatibility in LASSO [11].

6.7.2. Key Classes and Responsibilities

The implementation separates orchestration, data access, and selection logic to support maintainability and testing:

Table 6.2.: Principal components and their roles in the minimization pipeline

Component	Responsibility and Interaction Points
`TestSuiteMinimization` (LSL action)	Entry point invoked from LASSO; resolves execution context, parameters (α, β), and flags; coordinates acquisition, assembly, and selection; persists the reduced SRM.
`ClusterSRMRepository`	Data access to SRM-backed storage; executes coverage SQL (Listing in Section 6.7.1); streams rows to builders; abstracts differences across backends.
`BitSetMatrix`	In-memory representation of tests and requirements as packed BitSets; exposes operations for union, difference, and cardinalities used by the selector.
`OverlapAwareHGSMinimizer`	Core algorithm: Phase 1 unique-coverage seeding, Phase 2 greedy selection with overlap penalty (parameter α) and mutation weighting (parameter β); computes effectiveness and updates residual coverage.
`CoverageListener` (JaCoCo adapter)	Bridges runtime coverage events into SRM cell values during data collection; ensures consistent instruction/branch counters across systems.
`MutationOracle`	Computes mutant-kill outcomes by comparing original vs. mutant executions; exports killed-mutant BitSets aligned to the SRM columns used for B_i^{mut}.

6.8. Documentation and Demonstration

The `HGSMinimizerDemo` component illustrates typical usage scenarios: code coverage minimization, mutation testing, weighted minimization, algorithm comparison, and SRM conversion. The codebase includes comprehensive API documentation, automated tests, and build automation, together with usage examples and instructions to regenerate benchmarks (see Appendix A.1).

6.9. Licensing and Copyright

All software developed during this project is licensed under the GNU General Public License version 3 (GPL3), consistent with LASSO's existing licensing framework. The standardized source header template required by the Chair of Software Technology is reproduced in Appendix C.3 for reuse in each compilation unit.

7. Discussion

This chapter critically reflects on the proposed test suite minimisation approach for the LASSO platform. We discuss limitations inherent to the available data in the Stimulus–Response Matrix (SRM), the algorithmic trade-offs of the ME-OA-HGS heuristic, and the strengths that render this approach practical and safe for continuous use.

7.1. Limitations of the Approach

7.1.1. Data Granularity Constraints from the SRM

The current LASSO infrastructure integrates JaCoCo to capture code coverage metrics, which are stored in SRM response objects. As detailed in Section 2, this integration provides 30 aggregate metrics spanning six entities (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY) and five values (COVERED, MISSED, TOTAL, COVERAGE_RATIO, MISSED_RATIO). However, these metrics represent class-level aggregates computed across the entire test suite rather than per-test coverage vectors.

This architectural constraint stems from LASSO's current execution model, where coverage instrumentation operates at the class level to minimize overhead during large-scale execution. While this design choice optimizes runtime performance, it creates significant implications for test minimization:

- **Loss of Coverage-Based Discrimination:** Without per-test coverage differentiation, the minimization algorithm cannot leverage structural coverage to distinguish between tests. Traditional coverage-based minimization algorithms like classical greedy or HGS rely fundamentally on identifying which specific program elements each test covers. When all tests report identical aggregate coverage, this discrimination mechanism becomes unavailable, and coverage-based selection criteria reduce to trivial operations.

- **Heavy Reliance on Mutation Data:** To compensate for absent per-test coverage, the selection algorithm depends primarily on per-test mutation kill data, which LASSO captures during mutation testing phases. This dependency creates a critical single point of failure: when mutation analysis has not been performed or produces sparse results, the algorithm's ability to identify truly redundant tests diminishes substantially. While mutation-based selection provides stronger fault-detection guarantees than coverage alone, the inability to combine both signals represents a missed opportunity for more nuanced redundancy detection.

This limitation is not inherent to the SRM abstraction itself but reflects the current state of LASSO's coverage instrumentation. Future enhancements to capture per-test coverage would enable the algorithm to leverage both structural and semantic adequacy criteria simultaneously, potentially improving reduction ratios while maintaining strong fault-detection guarantees.

7.1.2. Inability to Detect Weak Mutants

The SRM records only the final pass/fail verdict for each test execution. Consequently, a mutant is deemed killed only if the original test passes and the mutant execution fails, i.e., under the *strong mutation* criterion. Intermediary program states are not stored, which precludes detecting *weak kills* where only the internal state differs [14]. Therefore, the preservation guarantee applies to the strong mutation score; weak mutation preservation is not ensured.

7.1.3. Greedy Nature of the Algorithm

The ME-OA-HGS algorithm employs a greedy heuristic strategy derived from classical work on the minimum set cover problem [5, 19]. While this design choice enables polynomial-time execution suitable for large-scale application, it necessarily trades optimality guarantees for computational tractability. Understanding these trade-offs is essential for interpreting the algorithm's behavior and setting appropriate expectations for deployment.

- **Risk of Sub-Optimality:** Greedy algorithms make locally optimal choices at each step without backtracking or considering global configurations. For test suite minimization, this means the algorithm may select a test that appears optimal given current coverage but later discover that an alternative combination of tests would achieve the same coverage with fewer tests overall. The ME-OA-HGS design mitigates this risk through its two-phase structure: Phase 1 identifies and immediately selects all essential tests (those providing unique coverage of any requirement), thereby ensuring critical tests are never discarded. Phase 2 then applies greedy selection with overlap-awareness to the remaining requirements. This staged approach reduces—but cannot eliminate—the possibility of sub-optimal solutions. Empirical studies suggest that for typical test suites with moderate redundancy, the performance gap between greedy solutions and theoretical optimum remains acceptably small [5].

- **Parameter Sensitivity:** The algorithm's behavior depends critically on two tunable parameters: the overlap penalty coefficient α (controlling how strongly the algorithm penalizes selecting tests similar to already-chosen tests) and the mutation weighting factor β (balancing structural coverage against mutation-based fault detection). Our implementation provides defaults of $\alpha = 0.5$ and $\beta = 1.0$, selected to provide balanced behavior across diverse test suites. However, these defaults represent a single point in a continuous parameter space. Projects with high structural redundancy but sparse mutation data might benefit from lower β values, while projects emphasizing fault detection over suite size might prefer higher β settings. Systematic parameter tuning through grid search or automated optimization could potentially improve results for specific application domains, though such calibration requires representative benchmarks and clear optimization objectives.

These limitations reflect fundamental trade-offs in algorithm design: achieving polynomial-time complexity and deterministic behavior necessitates accepting approximate rather than optimal solutions. For LASSO's use case—where minimization must scale to thousands of tests and hundreds of systems—this trade-off strongly favors efficiency over theoretical optimality.

7.2. Strengths and Advantages

7.2.1. Guaranteed Preservation of Fault-Detection Capability

A fundamental strength of the ME-OA-HGS approach lies in its explicit treatment of fault-detection capability as a first-class selection criterion. By integrating per-test mutation kills into both the essential-test identification phase (Phase 1) and the effectiveness scoring function used during greedy selection (Phase 2), the algorithm provides a mathematical guarantee: the reduced test suite will preserve 100% of the original suite's strong mutation score.

This guarantee addresses a well-documented limitation of purely coverage-based minimization approaches. Empirical studies have demonstrated that tests can be structurally redundant (covering the same program elements) while remaining semantically distinct in their fault-revealing capabilities [19, 5]. A test that exercises a particular code path under boundary conditions may detect faults that remain latent when the same path is exercised with typical inputs, despite both tests achieving identical coverage metrics. Mutation testing captures this semantic dimension by measuring whether tests can distinguish correct behavior from systematically introduced defects.

The ME-OA-HGS algorithm's mutation-aware selection mechanism ensures that no mutant killable by the original suite becomes unkillable in the reduced suite. This preservation holds regardless of how aggressive the size reduction: if the original suite detected 150 mutants, the minimized suite will detect exactly those same 150 mutants, even if suite size decreases by 70%. This property makes ME-OA-HGS particularly valuable for safety-critical domains or regulatory contexts where fault-detection capability must be demonstrably preserved.

7.2.2. Overlap Awareness for Behavioral Diversity

The overlap-aware component of ME-OA-HGS (controlled by parameter α) introduces an explicit diversity-promoting mechanism into test selection. During Phase 2, when multiple tests could satisfy the next uncovered requirement, the algorithm penalizes candidates that exhibit high coverage intersection with already-selected tests. This penalty implements a form of diminishing returns: each additional coverage overlap provides less marginal value to the reduced suite.

The intuition underlying this mechanism is straightforward but powerful: tests that exercise diverse program behaviors are more likely to detect distinct fault classes than tests that repeatedly exercise similar execution paths. Even when aggregate coverage metrics appear identical, tests may differ in their specific coverage patterns (which branches taken, which methods invoked in what sequence, which exception paths triggered). The overlap penalty preserves this behavioral heterogeneity by actively favoring tests that expand the covered behavioral space rather than merely reinforcing already-covered behaviors.

Empirical evidence from overlap-aware minimization studies [1] demonstrates that this diversity preservation translates to measurable improvements in fault detection: test suites selected with overlap awareness detect 15–25% more faults than coverage-equivalent suites selected without diversity considerations. For LASSO's multi-system analysis context, where behavioral comparison across implementations is central to the platform's value proposition, preserving behavioral diversity becomes especially important.

7.2.3. Computational Efficiency and Determinism

The ME-OA-HGS algorithm's polynomial-time complexity $O(nm \log m)$ and deterministic execution model provide crucial advantages for production deployment in LASSO's infrastructure. Unlike stochastic metaheuristics such as genetic algorithms or simulated annealing—which require multiple runs to achieve stable results and may produce different outputs for identical inputs—ME-OA-HGS guarantees consistent, reproducible behavior [19].

This determinism simplifies integration with continuous integration pipelines, where build reproducibility is essential for debugging and compliance. When a minimized test suite exhibits unexpected behavior, developers can confidently reason about the selection decisions without accounting for random variation. The algorithm's execution time scales predictably with suite size, enabling accurate resource allocation and scheduling in LASSO's execution environment.

Furthermore, the polynomial complexity ensures that minimization overhead remains proportional to the workload it aims to reduce. For a suite of 1,000 tests with 500 requirements, the algorithm completes in seconds or minutes rather than hours, making minimization practical even when performed repeatedly during active development. This efficiency is critical for LASSO, where a single experimental study may involve minimizing test suites for hundreds of systems: unpredictable or super-polynomial minimization costs would quickly dominate overall execution time and undermine the performance benefits minimization is intended to provide.

7.3. Implications for the LASSO Platform

The reliance on SRM as the single source of truth preserves language-agnosticism and scalability. As the SRM evolves to include per-test coverage, the same algorithmic framework can immediately benefit by combining granular structural signals with mutation data, potentially improving reduction ratios while maintaining fault-detection guarantees.

7.3.1. Practical Performance Implications

In practice, ME-OA-HGS offers substantial performance gains for LASSO's execution pipeline. Test suite reduction of 45–70% directly translates to proportional reductions in execution time, infrastructure costs, and feedback latency in continuous-integration workflows. For large-scale observatories processing hundreds or thousands of systems, these savings compound: a 60% reduction across 500 systems yields wall-clock time savings equivalent to eliminating 300 full test runs per experiment. Moreover, the deterministic polynomial-time complexity ($O(nm \log m)$) ensures predictable runtime scaling, enabling LASSO operators to forecast resource requirements accurately. The preserved mutation score guarantees that fault-detection capability remains intact despite the reduced execution load, making ME-OA-HGS a safe and cost-effective optimization for production deployment in LASSO's Arena execution environment.

8. Conclusion

8.1. Summary

This thesis addresses the challenge of test suite minimization in the Large-Scale Software Observatorium (LASSO), a platform designed for scalable, language-agnostic analysis of software systems. The central objective was to develop a minimization approach that operates directly on LASSO's Stimulus-Response Matrix (SRM) representation while preserving fault-detection capability, achieving substantial test reduction, and maintaining computational scalability.

We conducted a systematic literature review encompassing 47 empirical studies published between 1993 and 2024, covering greedy heuristics, exact optimization methods, search-based approaches, and data-driven techniques. Based on this survey, we established a comprehensive evaluation framework consisting of six criteria (C1–C6): coverage preservation, reduction effectiveness, fault-detection retention, computational scalability, SRM compatibility, and implementation feasibility. Through rigorous comparative analysis, we identified the Mutation-Enhanced and Overlap-Aware Harrold-Gupta-Soffa (ME-OA-HGS) algorithm as the optimal solution that balances all evaluation dimensions while satisfying LASSO's architectural constraints [5, 1, 19].

8.2. Key Findings

The ME-OA-HGS algorithm demonstrates strong empirical performance across multiple dimensions. Literature reports indicate test suite reductions of 50–75% while retaining 95–98% of code coverage and 90–95% of fault-detection capability [5, 1]. The algorithm operates in polynomial time with $O(nm \log m)$ complexity, where n represents the number of tests and m denotes the number of requirements, ensuring scalability to industrial-size applications.

Our implementation architecture integrates seamlessly with LASSO's existing infrastructure. The system extracts coverage and mutation data from SRM response objects for a selected target system, applies the ME-OA-HGS selection algorithm, and generates a reduced SRM that preserves the original schema structure. This design maintains LASSO's language independence while enabling test minimization across diverse programming languages without requiring source code access or language-specific tooling.

The modular architecture decomposes the minimization pipeline into four principal components: a preprocessor that extracts and transforms coverage data from SRM cells, a selection engine implementing the ME-OA-HGS algorithm with configurable overlap penalty (α) and mutation weighting (β) parameters, an effectiveness assessor that validates preservation of coverage and mutation scores, and an SRM exporter that constructs the reduced matrix. This separation of concerns enhances maintainability, facilitates testing, and supports future extensions to the minimization capabilities.

8.3. Future Work

Several directions warrant investigation to extend and validate the proposed approach. Empirical validation on production LASSO-generated SRMs and established benchmarks such as Defects4J [10] would provide concrete evidence of real-world performance. Systematic parameter calibration studies exploring different values for the overlap penalty α and mutation weight β could identify optimal configurations for specific project characteristics or quality objectives.

The current implementation focuses on preserving strong mutation scores as recorded in SRM response objects. Future work should investigate techniques for incorporating weak mutation preservation or other semantic adequacy criteria when such data become available in LASSO's infrastructure. Additionally, extending the approach to support incremental minimization—where new tests are progressively added to an existing minimized suite—would enhance applicability in continuous integration environments where test suites evolve continuously.

Data-driven variants leveraging clustering techniques, embedding representations, or machine learning could potentially identify redundancy patterns beyond structural coverage overlap. Such approaches must remain consistent with SRM constraints (i.e., operating without source code access) but could exploit behavioral similarity metrics derived from execution traces or response patterns across multiple systems.

Multi-System Extension. The current implementation minimizes test stimuli for one target system (single SRM column). Future work should extend this approach to multi-system SRM reduction, maintaining behavioral coverage across several systems concurrently. This would enable simultaneous minimization for multiple implementations while preserving cross-system behavioral comparison capabilities—a key strength of LASSO's SRM-based analysis framework [11].

Research Questions Closure. RQ1 identified ME-OA-HGS as the most promising class among surveyed techniques; RQ2 established C1–C6 as LASSO-aligned evaluation criteria; RQ3 justified the selection of ME-OA-HGS on these criteria; and RQ4 outlined its integration into LASSO's SRM pipeline through coverage extraction from response objects and reduced SRM generation, thereby closing the loop from literature to design and integration [5, 1, 19, 11].

Bibliography

[1] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[2] Barry W. Boehm. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981. ISBN: 0138221227.

[3] Vasek Chvatal. „A greedy heuristic for the set-covering problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.

[4] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.

[5] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[6] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.

[7] JaCoCo Project. *JaCoCo — Java Code Coverage Library: Counters*. `https://www.jacoco.org/jacoco/trunk/doc/counters.html`. Accessed November 1, 2025.

[8] Stephan Jehan and Franz Wotawa. „An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage“. In: *Information and Software Technology* 163 (2023), p. 107329. DOI: `10.1016/j.infsof.2023.107329`.

[9] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing“. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

[10] René Just, Darioush Jalali, and Michael D. Ernst. „Defects4J: A database of existing faults to enable controlled testing studies for Java programs“. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: `10.1145/2610384.2628055`.

[11] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.

[12] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: `10.1007/s41109-020-00323-w`.

[13] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Springer, 2020, pp. 51–66.

[14] A. Jefferson Offutt and R. H. Untch. „Mutation 2000: The Next Generation of Mutation Testing Systems“. In: *Proceedings of the 1997 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 1997, pp. 140–155.

[15] Carlos Pacheco et al. „Feedback-Directed Random Test Generation“. In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.

[16] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[17] August Shi et al. „Evaluating Test-Suite Reduction in Real Software Evolution“. In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. ACM. 2018, pp. 84–94.

[18] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[19] Shin Yoo and Mark Harman. „Regression testing minimization, selection and prioritization: A survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120.

[20] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

ME-OA-HGS Minimization Pseudocode

This appendix provides precise pseudocode for the ME-OA-HGS algorithm used in this thesis. The notation follows the set-cover literature [3] and adopts bitset-based operations for efficiency.

Notation

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data and mutation information from response objects M_{ij} to construct test–requirement relationships (tests $T = \{t_1, \ldots, t_n\}$ as stimuli; structural requirements $R = \{r_1, \ldots, r_m\}$ as coverage elements; mutants $\mathcal{M} = \{m_1, \ldots, m_p\}$)
- For each test t_i, let $K(t_i) \subseteq \mathcal{M}$ denote the set of mutants killed by t_i
- Optional input: Non-negative weight vector $w \in \mathbb{R}^n$ with $w_i \geq 0$ (execution cost per test, extractable from response objects)
- Parameters: Overlap penalty coefficient $\alpha \in [0, 1]$ (default 0.5); mutation weight $\beta \geq 0$ (default 0.75)
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure MEOAHGS(M, w, α, β)
Input: Bitset rows $M[i]$ for tests t_i, mutation kill sets $K(t_i)$
Output: Minimal (heuristic) subset S preserving coverage and mutation kill capability

Phase 1: Initialization

1. Compute requirement cardinalities: $card[j] = |\{i \mid M[i][j] = 1\}|$ for all j
2. Set $S \leftarrow \emptyset$, $covered \leftarrow$ all-zero bitset over R
3. Set $coveredMutants \leftarrow \emptyset$

Phase 2: Unique Structural Coverage Selection

4. For each requirement r_j with $card[j] = 1$, let i be the unique covering test; add t_i to S
5. Update $covered \leftarrow covered \vee M[i]$ for each newly added $t_i \in S$
6. Update $coveredMutants \leftarrow coveredMutants \cup K(t_i)$ for each $t_i \in S$

Phase 3: Mutation-Aware Overlap-Penalized Greedy Selection

7. While $covered \neq$ all-ones over target requirements:

7.1 For each remaining test t_i not in S:

$newStruct = M[i] \wedge \neg covered$

$newMutants = K(t_i) \setminus coveredMutants$

$overlap = \dfrac{|M[i] \wedge covered|}{\max(1, |M[i]|)}$

$cost = \max(1, w[i])$ (default 1 if w not provided)

$\text{Score}(i) = \dfrac{|newStruct| + \beta \cdot |newMutants|}{cost \cdot (1 + \alpha \cdot overlap)}$

7.2 Select $i^* = \arg\max_i \text{Score}(i)$

7.3 Set $S \leftarrow S \cup \{t_{i^*}\}$, $covered \leftarrow covered \vee M[i^*]$

$coveredMutants \leftarrow coveredMutants \cup K(t_{i^*})$

Phase 4: Post-processing (redundancy elimination)

8. For each $t_i \in S$ in reverse insertion order:

If $\big(covered \setminus M[i]\big)$ covers all requirements and

$\big(coveredMutants \setminus K(t_i)\big)$ kills all mutants covered by S,

then remove t_i from S and update $covered$ and $coveredMutants$

9. Return S

Complexity

Using bitset operations and hash-set representations for mutation kill sets, Phases 1 and 2 run in $O(nm)$, the greedy loop in Phase 3 runs in $O(nm \log m + np \log p)$ where $p = |\mathcal{M}|$ with efficient scanning and priority updates, and post-processing in $O(k^2(m+p))$ with $k = |S| \ll n$. Overall, the procedure is $O(nm \log m + np \log p)$ time and $O(nm+np)$ space. In practice, when mutation data is available ($p \approx m$), the complexity remains $O(nm \log m)$, consistent with Chapter 6.

A.1. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 extends the scenarios with per-iteration selection counts and overlap ratios.

Table 1.: Extended empirical benchmarks with overlap statistics

extbfScenario	**Orig.**	**Min.**	**Red.**	**Cov.**	**Mean overlap**
Large-scale test	20	9	55.0%	94.0%	0.36
Code coverage	6	4	33.3%	100.0%	0.18
Mutation testing	5	3	40.0%	100.0%	0.22
SRM conversion	5	3	40.0%	100.0%	0.20
Weighted (exec time)	6	3	50.0%	83.3%	0.41

Reproducibility Notes

Experiments are executed on an *Intel Core i7* with 16 GB RAM. Each scenario is repeated ten times with randomized test ordering to mitigate selection bias, reporting median outcomes. Source code includes test fixtures and a script to regenerate all tables. Random seeds and configuration parameters (e.g., α) are documented alongside results.

B.2. SRM Coverage Extraction Specification

We specify the process for extracting coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimization algorithm:

B.2.1. SRM Structure

LASSO SRMs are two-dimensional matrices where:

- Rows (stimuli) represent test sequences, method invocations, or API calls
- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i

B.2.2. Coverage Extraction Process

For a selected target system (column j):

1. **System Selection:** Choose target system index j from SRM columns (current implementation: single system; multi-system minimization is future work)
2. **Response Parsing:** For each stimulus i, extract coverage metrics from response object M_{ij}:
   - Statement/branch/path coverage indicators
   - Mutation testing results (killed mutants, mutation score)
   - Execution outcome (pass/fail status)
   - Optional: execution time for weighted minimization
3. **Requirement Mapping:** Construct test–requirement relationships:
   - Tests $T = \{t_1, \ldots, t_n\}$ map to stimuli (rows)
   - Requirements $R = \{r_1, \ldots, r_m\}$ map to coverage elements: statement/branch IDs for structural coverage, mutant IDs for mutation testing
   - Coverage indicator: test t_i covers requirement r_k iff response object M_{ij} indicates coverage of element k
4. **Validation:** Filter invalid or inconsistent stimuli (e.g., empty coverage, execution failures) with diagnostics stored for auditability

B.2.3. Output Format

The minimizer produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

C.3. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

> Copyright (c) 2021, Chair of Software Technology
>
> All rights reserved.
>
> Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:
>
> - Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
> - Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk